SUPERSYSTEMS PostgreSQL Study Guide & Solutions

Detailed Explanations & Reference Queries for 100 Data Engineer SQL Interview Questions

Part 1: Relational Schemas & Logic Explanations

• **Part Management (`part`)**: Registry for universal item codes (`part.parts_master` containing `part_number`), separator rules, category serial counters, and cross-reference catalog tables (`part.customer_mappings`).

• **Purchase & Procurement**: Requisition workflows (`purchase.purchase_requisitions`) and PO lead time tracker rules (`procurement.purchase_orders`) storing pricing, MOQ constraints, and promised dates.

• **Inventory (`inventory`)**: Tracks stock levels (`qty_on_hand`, `qty_reserved`, `qty_available`) and keeps an immutable transaction history ledger of all ISSUE, RECEIPT, or TRANSFER operations.

• **Warehouse (`warehouse`)**: Physical layouts including subdivisions like zones and bins (`bin_code`, `zone_code`, `capacity_units`) and receiving logs.

• **Quality Management (`quality`)**: QC inspection parameters (`quality.iqc_criteria`) and non-conformance records (`quality.ncrs`) generated automatically on rejections.

Example A: Raw Material Shortage & MOQ Calculations Query

```
SELECT
    b.item_code,
    b.qty_required - COALESCE(s.qty_available, 0) AS raw_shortage,
    GREATEST(
        b.qty_required - COALESCE(s.qty_available, 0),
        q.min_order_qty
    ) AS final_qty_to_order
FROM manufacturing.bom_lines b
LEFT JOIN inventory.stock_levels s ON b.item_code = s.part_number
LEFT JOIN purchase.supplier_quotations q ON b.item_code = q.part_or_rm_code
WHERE b.qty_required > COALESCE(s.qty_available, 0);
```

Part 2: 100 SQL Interview Questions & Answers

Q1: Star Schema vs Snowflake Schema? Snowflake for warehouse bin hierarchy?

Answer: Star Schema keeps dimensions denormalized (fewer joins, faster analytical queries). **Snowflake Schema** normalizes dimension tables into secondary tables (saves space, ensures referential integrity).
Warehouse Bin representation:
CREATE TABLE warehouse.zones (zone_code VARCHAR(50) PRIMARY KEY, zone_name VARCHAR(100));
CREATE TABLE warehouse.bins (bin_code VARCHAR(50) PRIMARY KEY, zone_code VARCHAR(50) REFERENCES warehouse.zones(zone_code));

Q2: Explain SCD Types 1, 2, and 3. Model price revisions in `purchase.supplier_quotations`.

Answer: Type 1 overwrites values. **Type 2** creates new rows with validity timestamps (`valid_from`, `valid_to`) and an active flag to preserve history. **Type 3** uses columns to store the previous state.

Type 2 configuration:
CREATE TABLE purchase.supplier_quotation_history (id UUID, part_or_rm_code VARCHAR(100), supplier_code VARCHAR(50), unit_price NUMERIC(18,2), valid_from TIMESTAMP, valid_to TIMESTAMP, is_current BOOLEAN);

Q3: Surrogate key vs Natural key?

Answer: A **Natural key** uses pre-existing business variables (e.g. `part_number`). A **Surrogate key** is a synthetic key (e.g. UUID, Auto-increment) with no business value. Surrogate keys protect the database schema from real-world attribute changes.

Q4: Many-to-many relationship modeling.

Answer: Model many-to-many relationships by creating a junction table storing foreign key references:
CREATE TABLE project.project_purchase_orders (project_id VARCHAR(36) REFERENCES project.projects(id), po_id VARCHAR(36) REFERENCES procurement.purchase_orders(id), PRIMARY KEY (project_id, po_id));

Q5: Explain normal forms (1NF, 2NF, 3NF, BCNF). jsonb `lines` in POs?

Answer: 1NF: atomic values, no arrays. Storing repeating nested structures inside JSONB column violates 1NF. **2NF**: 1NF + no partial key dependency. **3NF**: 2NF + no transitive dependency. **BCNF**: every determinant is a superkey. JSONB saves migrations but compromises foreign key validation.

Q6: What is denormalization and when is it preferred?

Answer: Adding redundancy to database schemas to avoid expensive join queries. Highly preferred in read-heavy database architectures (OLAP, Data Warehousing, analytics reporting).

Q7: Audit logging without triggers.

Answer: Implement audits at the application layer via ORM event listeners (e.g. SQLAlchemy pre-commit events) or use Log-based Change Data Capture (CDC) like Debezium parsing Postgres Write-Ahead Logs (WAL).

Q8: UUIDs vs Auto-increment BigInts in PostgreSQL.

Answer: UUIDs: Globally unique, safe from enumeration. But their large size (128-bit) and non-sequential nature cause page splits and fragment indexes. **BigInts**: Fast indexing, sequential, but expose record count.

Q9: Hierarchical parent-child modeling in accounts.

Answer: Use an adjacency model with a recursive parent foreign key:
CREATE TABLE finance.accounts (id VARCHAR(36) PRIMARY KEY, name VARCHAR(200), parent_id VARCHAR(36) REFERENCES finance.accounts(id));

Q10: Table partitioning by range.

Answer: Divides a large table into smaller physical tables based on value ranges:
CREATE TABLE inventory.stock_movements (id UUID, created_at TIMESTAMP NOT NULL) PARTITION BY RANGE (created_at);
CREATE TABLE inventory.stock_movements_2026_07 PARTITION OF inventory.stock_movements FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');

Q11: Write-Ahead Log (WAL)?

Answer: A log file on disk where changes are written *before* updating actual database pages. Ensures transaction durability during power crashes.

Q12: Cascade vs Set Null on delete?

Answer: ON DELETE CASCADE deletes all child records automatically when parent rows are deleted. **ON DELETE SET NULL** updates child foreign key values to NULL, keeping child records alive.

Q13: Database sharding vs partitioning?

Answer: Partitioning divides tables physically within the *same* database instance. **Sharding** distributes tables across *multiple* database servers/nodes.

Q14: JSON vs JSONB in PostgreSQL.

Answer: JSON stores exact text formatting (faster writes, slow reads). **JSONB** stores compiled binary format (slower writes, very fast reads, supports indexing).

Q15: GIN index vs B-Tree on JSONB.

Answer: GIN indexes every key and value inside the JSONB structure (flexible, supports containment query `@>`). **B-Tree** only indexes a specific key path, which is faster and smaller but less flexible.

Q16: Describe the difference between OLTP and OLAP database engines. Which category does this SUPERSYSTEMS PostgreSQL instance fit into?

Answer: OLTP: designed for high-concurrency transactional row mutations (ERP). **OLAP**: designed for fast aggregations on column chunks (data warehouse). SUPERSYSTEMS is OLTP.

Q17: Explain the Entity-Attribute-Value (EAV) design pattern. What are its pros and cons?

Answer: Stores metadata fields as rows (Entity, Attribute, Value). Pro: Infinite schema flexibility without DDL modifications. Con: Requires complex self-joins to query single rows, causing slow queries.

Q18: How would you model user roles and permissions (RBAC) like the `iam.module_access` table to allow granular column-level control?

Answer: Map permissions in a matrix table using user, resource, and permission columns:
CREATE TABLE iam.module_access (user_id VARCHAR(36) REFERENCES iam.users(id), module VARCHAR(100), role VARCHAR(50), PRIMARY KEY(user_id, module));

Q19: Multi-tenancy: Shared DB vs Schema-per-tenant?

Answer: Shared DB + tenant_id: Low cost, simple maintenance, but carries risk of cross-tenant leaks. **Schema-per-tenant**: High security isolation, but high resource overhead and hard to execute DDL migrations.

Q20: What is a metadata schema? How would you design a table to dynamically define UOM conversions across different modules?

Answer: A configuration lookup table:
CREATE TABLE master.uom_conversions (from_uom VARCHAR(10), to_uom VARCHAR(10), factor NUMERIC(18,6), PRIMARY KEY(from_uom, to_uom));

Q21: How does a B-Tree index work under the hood?

Answer: A self-balancing search tree. Stores keys in sorted order. Searches traverse logarithmic paths from root node to branch nodes and reach leaf nodes containing row pointers.

Q22: Explain the difference between Clustered and Non-Clustered indexes.

Answer: Clustered defines the physical sequential order of rows on disk (only one allowed). **Non-Clustered** is a separate pointer structure (multiple allowed).

Q23: What is a Composite Index? Why does the order of columns in a composite index matter?

Answer: An index on multiple columns. Order matters because queries search from left-to-right. An index on `(tenant_id, part_number)` cannot optimize queries filtering *only* on `part_number`.

Q24: How do you identify slow-running queries in PostgreSQL? Explain the configuration parameters like `pg_stat_statements`.

Answer: Configure `pg_stat_statements` in `postgresql.conf`, set `log_min_duration_statement` to log slow queries, and monitor active sessions in `pg_stat_activity`.

Q25: Explain the output of `EXPLAIN ANALYZE`. What is the difference between a Sequential Scan, Index Scan, and Index Only Scan?

Answer: Seq Scan reads every page (slow). **Index Scan** reads the index and then table pages. **Index Only Scan** reads the index pages directly without table lookups (fastest).

Q26: What is a Index Only Scan, and what conditions are required to trigger it?

Answer: Triggered when the query SELECTs and FILTERs on columns that are part of the index, and the table's visibility map is vacuumed.

Q27: What is index selectivity, and how does it impact the query planner's decision to use an index?

Answer: Ratio of distinct values to total rows. High selectivity (unique email) utilizes indexes. Low selectivity (boolean flags) defaults to Seq Scan.

Q28: Explain what database statistics are and why running `ANALYZE` regularly is critical for query optimizer performance.

Answer: Statistics are data catalogs containing value histograms and frequency distributions. Running `ANALYZE` keeps these stats updated so the planner can select cost-effective query plans.

Q29: What is Index Bloat? How do you detect and resolve index bloat in PostgreSQL?

Answer: Empty slots left inside index leaf nodes caused by updates/deletions. Reclaim space by executing `REINDEX INDEX index_name` or `VACUUM`.

Q30: Explain the difference between `VACUUM`, `VACUUM FULL`, and `autovacuum`.

Answer: VACUUM marks dead tuples as reusable space (no locks). **VACUUM FULL** shrinks physical disk size by rewriting tables (locks table). **Autovacuum** background daemon automate this process.

Q31: What is a Partial Index? Write a query to create a partial index on `inventory.stock_levels` for rows where `qty_available <= reorder_point`.

Answer: An index created on a filtered subset of rows:
CREATE INDEX idx_part_low_stock ON inventory.stock_levels (part_number) WHERE qty_available <= reorder_point;

Q32: What is an Expression Index? How would you index case-insensitive email lookups in `iam.users`?

Answer: Indexes query values computed by functions:
CREATE INDEX idx_user_lower_email ON iam.users (LOWER(email));

Q33: Explain the term "Parameter Sniffing" and how it affects query plan caching.

Answer: The engine compiles query plans based on parameter values parsed at first compilation, leading to inefficient execution plans if parameter data density fluctuates.

Q34: What is a Hash Join, Merge Join, and Nested Loop? In what conditions does the query planner choose one over another?

Answer: Nested Loop: best for small datasets with index lookups. **Hash Join**: best for large, unsorted sets (builds in-memory hash table). **Merge Join**: best for pre-sorted inputs.

Q35: How does the size of the database buffer cache (`shared_buffers` in PG) affect index scan performance?

Answer: Larger buffer cache enables indexes to reside in memory, eliminating random disk read I/O operations.

Q36: How do you optimize queries that perform large `LIMIT` and `OFFSET` pagination? Suggest alternative cursor-based pagination strategies.

Answer: OFFSET requires scanning and discarding rows. Use keyset pagination:
SELECT id FROM procurement.purchase_orders WHERE created_at > :last_seen_date ORDER BY created_at LIMIT 20;

Q37: What is a covering index, and how does the `INCLUDE` clause help in index-only scans?

Answer: An index that appends non-key payload values to leaf nodes, enabling index-only scans:
CREATE INDEX idx_covering ON inventory.stock_levels(part_number) INCLUDE (qty_available);

Q38: Explain what happens during a table scan when a table is heavily fragmented.

Answer: Disk head performs random read seeking instead of contiguous sequential block reads, slowing down table scans.

Q39: How do you tune queries that contain complex OR conditions in their `WHERE` clause to ensure they utilize indexes?

Answer: Split OR filters into `UNION` or `UNION ALL` blocks so each select statement evaluates its own index path separately.

Q40: Explain the difference between active indexing and writing indexes on write-heavy tables.

Answer: Indexes slow down write/insert/update execution speed because index structures must be maintained synchronously on every row mutation.

Q41: What are Window Functions? Write a query using `ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()` to rank products by inventory stock value.

Answer: Window functions compute values across rows partition groupings without collapsing them.
SELECT part_number, ROW_NUMBER() OVER (ORDER BY qty_on_hand*unit_cost DESC) AS row_num, RANK() OVER (ORDER BY qty_on_hand*unit_cost DESC) AS rnk, DENSE_RANK() OVER (ORDER BY qty_on_hand*unit_cost DESC) AS dense_rnk FROM inventory.stock_levels;

Q42: Write a query to calculate a 3-month moving average of total purchase order amounts from `procurement.purchase_orders`.

Answer: SELECT date_trunc('month', date) AS mo, SUM(total) AS sum_tot, AVG(SUM(total)) OVER (ORDER BY date_trunc('month', date) ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) FROM procurement.purchase_orders GROUP BY 1;

Q43: How do you find the running total of stock movements for a specific item over time using window functions?

Answer: SELECT part_number, created_at, SUM(CASE WHEN movement_type='RECEIPT' THEN qty ELSE -qty END) OVER (PARTITION BY part_number ORDER BY created_at) FROM inventory.stock_movements;

Q44: Explain the difference between `ROWS BETWEEN` and `RANGE BETWEEN` in window function frame specifications.

Answer: ROWS evaluates offsets as physical row offsets. **RANGE** evaluates offsets as value-based offsets on the order key (e.g. date intervals).

Q45: Write a recursive CTE query to display the entire hierarchy of accounts starting from the root parent in `finance.accounts`.

Answer: WITH RECURSIVE acc_tree AS (SELECT id, name, parent_id FROM finance.accounts WHERE parent_id IS NULL UNION ALL SELECT a.id, a.name, a.parent_id FROM finance.accounts a INNER JOIN acc_tree t ON a.parent_id = t.id) SELECT * FROM acc_tree;

Q46: Write a recursive CTE query to resolve a multi-level Bill of Materials (BOM) explosion, aggregating components needed for a parent finished good.

Answer: WITH RECURSIVE bom_exp AS (SELECT parent_part, component_part, qty_required FROM manufacturing.bom_lines WHERE parent_part=:fg UNION ALL SELECT b.parent_part, b.component_part, b.qty_required*e.qty_required FROM manufacturing.bom_lines b INNER JOIN bom_exp e ON b.parent_part=e.component_part) SELECT component_part, SUM(qty_required) FROM bom_exp GROUP BY 1;

Q47: What is a Common Table Expression (CTE)? When is a CTE preferred over a subquery, and what is its performance impact in PostgreSQL (materialized vs non-materialized)?

Answer: A temporary named result dataset. Preferred for readability. By default, PG materializes reusable CTEs, but adding `NOT MATERIALIZED` tells the engine to inline them directly into the execution plan.

Q48: What are `LEAD()` and `LAG()` window functions? Write a query to find the lead time difference in days between consecutive purchase orders for each supplier.

Answer: SELECT supplier_code, promised_delivery_date - LAG(promised_delivery_date) OVER (PARTITION BY supplier_code ORDER BY created_at) FROM purchase.purchase_orders;

Q49: Write a query using `COALESCE` and `NULLIF` to prevent division-by-zero errors when calculating the order fill rate (received qty / requested qty).

Answer: SELECT received_qty / NULLIF(requested_qty, 0) FROM purchase.purchase_requisitions;

Q50: Explain `PIVOT` and `UNPIVOT` concepts. How would you pivot the weekly total purchase order counts by status in PostgreSQL using `FILTER` clauses?

Answer: SELECT date_trunc('week', date) AS wk, COUNT(*) FILTER (WHERE status='open') AS open_tot, COUNT(*) FILTER (WHERE status='closed') AS closed_tot FROM procurement.purchase_orders GROUP BY 1;

Q51: What is the difference between `UNION` and `UNION ALL`? Explain their memory and performance characteristics.

Answer: UNION deduplicates records (requires sorting/hashing, slower). **UNION ALL** merges rows without checking for duplicates (faster, low memory).

Q52: Write a query to identify top 5% of customers by total invoice amount using `NTILE()`.

Answer: WITH bucket_invoices AS (SELECT party_id, SUM(total) AS tot, NTILE(20) OVER (ORDER BY SUM(total) DESC) AS tile FROM finance.invoices GROUP BY 1) SELECT party_id FROM bucket_invoices WHERE tile=1;

Q53: How do you perform fuzzy string matching in SQL? Explain `LIKE`, `ILIKE`, Levinshtein distance, and Trigrams.

Answer: LIKE/ILIKE uses basic wildcards. **Levenshtein** calculates minimum character edits. **Trigram** splits words into 3-character slices to compare character overlap.

Q54: Write a query to group and aggregate sales data using `GROUP BY CUBE` and explain its difference from `GROUP BY ROLLUP` and `GROUP BY GROUPING SETS`.

Answer: ROLLUP calculates hierarchical subtotals. CUBE calculates all permutation subtotal combinations. GROUPING SETS specifies explicit grouping arrays.

Q55: Explain the execution order of clauses in a SQL statement.

Answer: Order: `FROM` → `JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `WINDOW` → `DISTINCT` → `UNION` → `ORDER BY` → `LIMIT`.

Q56: What is a correlated subquery? Write a query to find all employees whose salary is above the average salary of their department.

Answer: A subquery that references columns from the outer query:
SELECT first_name, salary FROM hr.employees e WHERE salary > (SELECT AVG(salary) FROM hr.employees WHERE department_id=e.department_id);

Q57: Write a query to extract, parse, and aggregate elements from the JSONB column `lines` in the `procurement.purchase_orders` table.

Answer: SELECT line_item->>'part_number' AS p, SUM((line_item->>'quantity')::numeric) FROM procurement.purchase_orders, LATERAL jsonb_array_elements(lines) AS line_item GROUP BY 1;

Q58: Explain the difference between `CROSS JOIN` and `INNER JOIN`. In what real-world analytics cases is a cross join useful?

Answer: CROSS JOIN creates a Cartesian product (all combinations). **INNER JOIN** matches rows on condition. Cross joins are useful to generate base reporting metrics like calendar-product grids.

Q59: How would you write a query to find the median value of purchase order totals without using built-in median functions in databases that do not support it?

Answer: SELECT AVG(total) FROM (SELECT total, ROW_NUMBER() OVER (ORDER BY total) AS r, COUNT(*) OVER () AS c FROM procurement.purchase_orders) x WHERE r IN ((c+1)/2, (c+2)/2);

Q60: Write a query to identify overlapping project task schedules in `project.tasks` using interval logic.

Answer: SELECT a.id, b.id FROM project.tasks a INNER JOIN project.tasks b ON a.project_id=b.project_id AND a.id < b.id WHERE (a.start_date, a.end_date) OVERLAPS (b.start_date, b.end_date);

Q61: What is the difference between `ANY`, `ALL`, and `SOME` operators in subquery evaluations?

Answer: ANY (or SOME) returns true if comparison is true for at least one record. **ALL** returns true only if true for all records.

Q62: Write a query using `EXISTS` vs `IN` to check if supplier parts have active contracts. Explain the performance differences.

Answer: SELECT * FROM supplier.suppliers s WHERE EXISTS (SELECT 1 FROM supplier.contracts c WHERE c.supplier_code=s.supplier_code); EXISTS is faster for correlated lookups since it halts execution on the first match.

Q63: Explain the difference between `NULL` and empty string `''`. How does SQL evaluate expressions like `NULL = NULL` and `NULL IS NULL`?

Answer: Empty string is a known blank value. NULL is an unknown state. `NULL = NULL` yields `NULL`. Use `IS NULL` to evaluate safely.

Q64: Write a query to calculate the Year-Over-Year (YoY) growth of invoice amounts.

Answer: WITH sales AS (SELECT date_part('year', date) AS y, SUM(total) AS t FROM finance.invoices GROUP BY 1) SELECT y, (t - LAG(t) OVER (ORDER BY y)) / LAG(t) OVER (ORDER BY y) * 100 FROM sales;

Q65: What is the `LATERAL` join operator in PostgreSQL? Write a query using `LEFT JOIN LATERAL` to fetch the 3 most recent stock movements for each warehouse bin.

Answer: A subquery join that references columns of preceding tables:
SELECT b.bin_code, m.* FROM warehouse.bins b LEFT JOIN LATERAL (SELECT movement_no FROM inventory.stock_movements WHERE from_bin_code=b.bin_code ORDER BY created_at DESC LIMIT 3) m ON TRUE;

Q66: How do you implement date-binning or grouping by arbitrary time intervals (e.g., every 15 minutes) for timeseries data?

Answer: Use timescaledb `time_bucket('15 minutes', created_at)` or division arithmetic on Unix timestamps in vanilla SQL.

Q67: Explain the function of the `CASE` statement. Write a query categorizing stock levels into 'Out of Stock', 'Reorder Alert', and 'Healthy' buckets.

Answer: SELECT part_number, CASE WHEN qty_available <= 0 THEN 'Out of Stock' WHEN qty_available <= reorder_point THEN 'Reorder Alert' ELSE 'Healthy' END FROM inventory.stock_levels;

Q68: Write a query to find the first non-null contact method (mobile, landline, email) for employees.

Answer: SELECT first_name, COALESCE(phone, email, 'No contact info') FROM hr.employees;

Q69: What is a self-join? Write a query to list all employees and the names of their direct managers from `hr.employees`.

Answer: A table joined with itself:
SELECT emp.first_name, mgr.first_name FROM hr.employees emp LEFT JOIN hr.employees mgr ON emp.manager_id=mgr.id;

Q70: Write a query to identify gaps in consecutive document sequences (e.g., missing numbers in invoice document sequence `INV-001`, `INV-003`).

Answer: Generate a numeric sequence using `generate_series` and left-join your table to find records where the join result is NULL.

Q71: What does the ACID acronym stand for? Explain each concept in detail.

Answer: Atomicity (all-or-nothing), **Consistency** (valid state transitions), **Isolation** (concurrency control), and **Durability** (committed data persists).

Q72: What are the four ANSI SQL transaction isolation levels? What concurrency anomalies does each prevent?

Answer: 1. **Read Uncommitted** (none), 2. **Read Committed** (prevents dirty reads), 3. **Repeatable Read** (prevents dirty + non-repeatable reads), 4. **Serializable** (prevents all anomalies including phantom reads).

Q73: What is Read Committed isolation level? Explain how dirty reads are avoided.

Answer: Default level. Transaction readers only see data committed prior to starting. Dirty reads are avoided by checking the commit log and ignoring active/uncommitted rows.

Q74: Explain MVCC (Multi-Version Concurrency Control) in PostgreSQL. How does it manage concurrent reads and writes without locking?

Answer: Postgres writes a new row version (tuple) instead of overwriting. Readers see a consistent snapshot based on their start transaction ID without blocking active writes.

Q75: What is a Deadlock? How does a database engine detect and resolve deadlocks, and how can you write queries to prevent them?

Answer: Two transactions blocking each other, waiting for locks. Database breaks the loop by aborting one transaction. Prevent by acquiring locks in a consistent table order.

Q76: What is the difference between Shared Locks (S) and Exclusive Locks (X)?

Answer: Shared (S) allows multiple concurrent reads. **Exclusive (X)** blocks both concurrent reads and writes (used by updates/deletions).

Q77: Explain the use of the `FOR UPDATE` clause in a SELECT query. When should you use `FOR UPDATE SKIP LOCKED`?

Answer: Locks queried rows. Use `SKIP LOCKED` in multi-worker queues so workers process available jobs immediately without blocking on locked rows.

Q78: What is Write-Ahead Logging (WAL) and how does it guarantee transaction durability?

Answer: Writes transaction logs to disk before modifying actual table data pages, ensuring transaction state recovery during crashes.

Q79: Explain the difference between a database transaction `COMMIT` and `ROLLBACK` at the disk/page level.

Answer: COMMIT marks transaction ID as committed in pg_xact and flushes WAL. **ROLLBACK** marks it as aborted (the garbage collector removes aborted row versions).

Q80: Describe SQL Injection and write a code example showing how to prevent it using parameterized queries.

Answer: Malicious SQL input executed dynamically. Prevent by using parameter bindings:
db.session.execute(text('SELECT * FROM users WHERE email=:e'), {'e': user_email});

Q81: What is Role-Based Access Control (RBAC) vs Row-Level Security (RLS) in PostgreSQL? How would you write an RLS policy for the `inventory.stock_levels` table to restrict records by `tenant_id`?

Answer: RBAC grants permissions by tables/roles. **RLS** filters specific rows dynamically:
CREATE POLICY tenant_pol ON inventory.stock_levels USING (tenant_id = current_setting('app.tenant_id'));

Q82: What are Database Triggers? Explain the difference between `BEFORE` and `AFTER` triggers, and row-level vs statement-level triggers.

Answer: Stored procedures triggered on DML operations. **BEFORE** triggers validate/modify row values. **AFTER** triggers log audit trials to separate tables.

Q83: What is a savepoint inside a transaction? Write a transaction script using savepoints to partially roll back changes on validation failure.

Answer: A subset commit boundary inside a transaction:
SAVEPOINT my_save; INSERT INTO tbl ...; ROLLBACK TO SAVEPOINT my_save; COMMIT;

Q84: Explain the difference between `TRUNCATE` and `DELETE`.

Answer: DELETE scans and removes rows individually (slower, writes WAL, triggers fire). **TRUNCATE** deallocates table pages (fast, bypasses triggers, needs exclusive lock).

Q85: How do you handle schema upgrades in production databases safely without causing table locks and downtime?

Answer: Add columns as nullable, build indexes concurrently (`CREATE INDEX CONCURRENTLY`), and update values in small batched transactions.

Q86: What is the difference between ETL and ELT? When is one preferred over the other?

Answer: ETL transforms data before writing (ideal for security, small storage). **ELT** loads raw data and utilizes the data warehouse to transform it (ideal for scalability).

Q87: What is Change Data Capture (CDC)? How does log-based CDC differ from query-based CDC?

Answer: Query-based queries fields like `updated_at` (misses hard deletes, slows tables). **Log-based** parses WAL logs directly (real-time, zero impact, captures deletes).

Q88: Explain the concept of Idempotency in data pipelines. Why is it important, and how do you design an idempotent insert pipeline?

Answer: Pipelines produce identical results regardless of executions. Design using primary/unique keys and upsert instructions (`ON CONFLICT DO NOTHING`).

Q89: How do you handle backfills of historical data in a production data pipeline without impacting live transaction tables?

Answer: Break backfills into small batch partition queries (e.g., daily chunks) during off-peak hours to avoid query timeouts.

Q90: Explain the difference between Batch processing and Stream processing.

Answer: Batch processes historical data blocks periodically. **Stream** processes events in real-time as they are written (e.g. Kafka pipeline).

Q91: What is Data Lineage, and why is it crucial for data engineering platforms?

Answer: A dependency chart showing how data flows from sources to models. Critical for impact assessment, data auditing, and pipeline debugging.

Q92: How do you handle late-arriving data in a time-series data warehouse?

Answer: Re-evaluate aggregate partitions for the delayed date range rather than appending them to the current run timestamp.

Q93: What is the difference between a database view, a materialized view, and a table?

Answer: View is a dynamic query reference. **Materialized View** caches query results physically on disk. **Table** is the primary physical storage.

Q94: Explain the concept of a Staging Area in ETL architecture.

Answer: A landing database zone where raw source data is cleaned and validated before loading into target warehouse dimension tables.

Q95: How do you handle schema evolution (e.g., columns being added or data types changing) in upstream sources without breaking downstream tables?

Answer: Utilize schema registry systems to enforce compatibility standards, or map columns inside flexible JSONB fields.

Q96: Explain what a surrogate key is in a Data Warehouse. Why is it preferred over natural keys for business reporting?

Answer: A synthetic primary key that isolates the warehouse schema from changes in source database natural key attributes.

Q97: What is Upsert? Write a PostgreSQL SQL statement using `INSERT ... ON CONFLICT DO UPDATE` to upsert stock levels.

Answer: INSERT INTO inventory.stock_levels (part_number, qty_on_hand, tenant_id) VALUES ('P1', 100, 'T1') ON CONFLICT (part_number, tenant_id) DO UPDATE SET qty_on_hand=EXCLUDED.qty_on_hand, updated_at=NOW();

Q98: What is the role of metadata databases (e.g., Apache Airflow DB) in orchestrating data workflows?

Answer: Stores execution parameters, dependency task mappings, retries, and run metrics.

Q99: Explain how you would monitor database query performance trends over time using ELK stack or Prometheus/Grafana.

Answer: Configure prometheus exporter (e.g., `postgres_exporter`) to parse metrics from `pg_stat_statements` and build Grafana alert thresholds.

Q100: How do you design a data validation pipeline to check for referential integrity, null values, and outliers before loading data into a BI data warehouse?

Answer: Run validation queries on staging tables checking for orphans and nulls, aborting the write pipeline on violation alerts.